

Partie 1 — Enrichissement de données

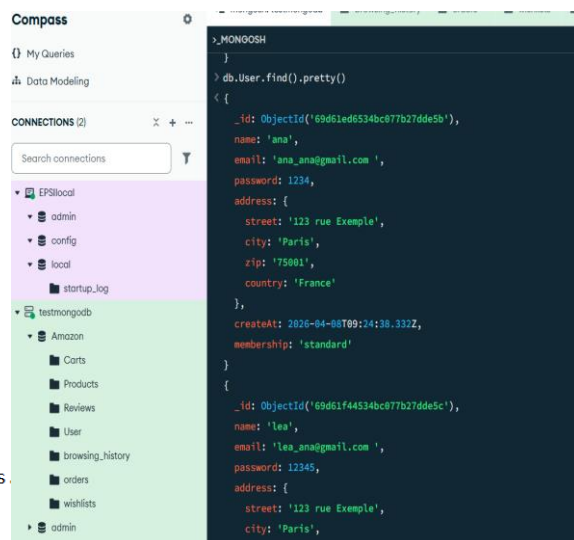
Exercice 1.1 : Ajout de champs utilisateur

L'objectif est d'enrichir chaque document de la collection users avec deux nouveaux champs : une adresse structurée (address) et un niveau d'abonnement (membership).

J'ai choisi de stocker l'**adresse** directement dans le document utilisateur sous forme de sous-document, et non dans une collection séparée. Ce choix repose sur le fait que l'adresse est toujours consultée en même temps que le profil utilisateur. Il n'existe aucun cas d'usage où l'on aurait besoin de l'adresse sans l'utilisateur. La relation est de type 1:1, ce qui justifie pleinement l'embedding.

```
db.User.updateMany(
  {}, //
  {
    $set: {
      address: {
        street: "123 rue Exemple",
        city: "Paris",
        zip: "75001",
        country: "France"
      }
    }
  }
)

//updateMany({}) → met à jour tous les documents.
//$set → ajoute ou modifie un champ.
```



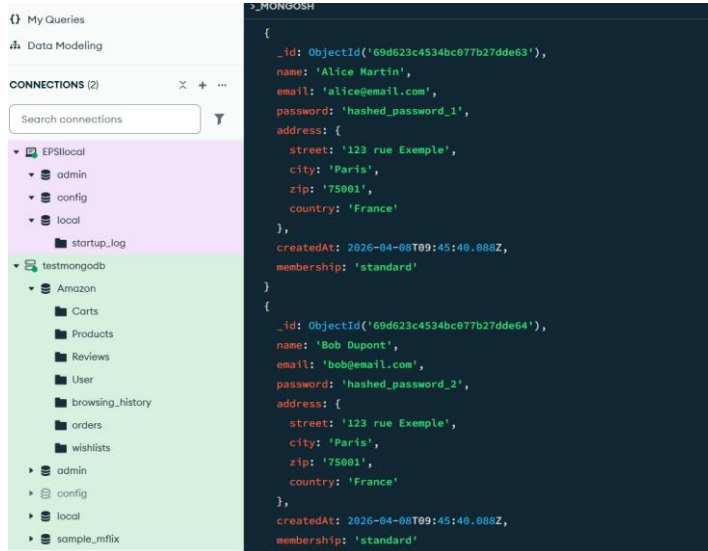
1. Ajouter un champ membership qui vaut "premium" pour 30 % des utilisateurs et "standard" sinon.

J'ai ajouté un champ simple de type string avec deux valeurs possibles : premium ou standard. Ce champ ne nécessite pas de collection dédiée car il n'a pas de données propres — c'est uniquement une étiquette métier. La distribution 30% premium / 70% standard a été simulée avec `Math.random() < 0.3` pour reproduire une répartition réaliste dans une plateforme e-commerce.

```
db.User.find({}, { _id: 1 }).forEach(user => {
  const membership = Math.random() < 0.3 ? "premium" : "standard";
  db.User.updateOne(
    { _id: user._id },
    { $set: { membership: membership } }
  );
});

//Math.random() < 0.3 → 30% de chance d'obtenir "premium".
```

résultat :



Exercice 1.2 : Simuler l'historique de commandes

Un historique de commandes a été simulé dans la collection orders.

```
db.createCollection("orders");

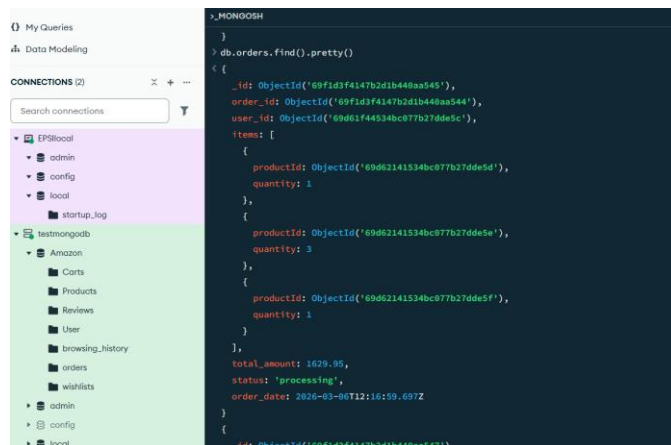
db.Carts.aggregate([
  {
    $lookup: {
      // $lookup : fait une jointure entre Carts et Products pour récupérer les informations des produits.
      from: "Products",
      localField: "items.productId",
      foreignField: "_id",
      as: "productDetails"
    }
  }
]).forEach(cart => {

  // Calcul du total
  let total = 0;
  cart.items.forEach(item => {
    const product = cart.productDetails.find(p =>
      p._id.toString() === item.productId.toString()
    );
    if (product) {
      total += product.price * item.quantity;
    }
  });

  // Génération d'une date aléatoire (3 derniers mois)
  const now = new Date();
  const threeMonthsAgo = new Date();
  threeMonthsAgo.setMonth(now.getMonth() - 3);
  const randomDate = new Date(
    threeMonthsAgo.getTime() + Math.random() * (now.getTime() - threeMonthsAgo.getTime())
  );

  // Insertion de la commande
  db.orders.insertOne({
    order_id: new ObjectId(),
    user_id: cart.user_id,
    items: cart.items,
    total_amount: Math.round(total * 100) / 100,
    status: "processing",
    order_date: randomDate
  });
});
```

résultat :



Partie 2 — Nouvelles collections et relations

Exercice 2.1 : Wishlist

Récupère tous les produits et utilisateurs et pour chaque utilisateur : choisit aléatoirement entre 5 et 10 produits et crée une wishlist avec ces produits insère le tout dans la collection wishlists .

J'ai eu seulement 3 produits et j'ai ajoutée 7 produits avec `=> db.Products.insertMany([ ...`

et après :

```
//récupère tous les produits et utilisateurs
//pour chaque utilisateur :
//choisit aléatoirement entre 5 et 10 produits
//crée une wishlist avec ces produits
//insère le tout dans la collection wishlists

const productIds = db.Products.find({}, {_id: 1}).toArray().map(p => p._id)

db.User.find({}, {_id: 1}).toArray().forEach(user => {
  const count = Math.floor(Math.random() * 6) + 5 // entre 5 et 10
  const shuffled = [...productIds].sort(() => Math.random() - 0.5)
  db.wishlists.insertOne({
    user_id: user._id,
    products: shuffled.slice(0, count)
  })
})

//sort(() => Math.random() - 0.5) → mélange le tableau aléatoirement. slice(0, count) → prend N éléments.
résultat :
```

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

- EPSIlocal
 - admin
 - config
 - local
 - startup_log
- testmongodb
 - Amazon
 - Carts
 - Products
 - Reviews
 - User
 - browsing_history
 - orders
 - wishlists
 - admin
 - config
 - local
 - sample_mflix

```
> mongosh: testmongodb
> _MONGOSH
>
> db.wishlists.find().pretty()
< {
  _id: ObjectId('69f4bf41562377cale058331'),
  user_id: ObjectId('69d61ed6534bc077b27dde5b'),
  products: [
    ObjectId('69f4bee7562377cale05832c'),
    ObjectId('69f4bee7562377cale05832e'),
    ObjectId('69d62141534bc077b27dde5e'),
    ObjectId('69f4bee7562377cale058330'),
    ObjectId('69d62141534bc077b27dde5f'),
    ObjectId('69f4bee7562377cale05832f'),
    ObjectId('69d62141534bc077b27dde5d'),
    ObjectId('69f4bee7562377cale05832d'),
    ObjectId('69f4bee7562377cale05832b'),
    ObjectId('69f4bee7562377cale05832a')
  ]
}
{
  _id: ObjectId('69f4bf41562377cale058332'),
  user_id: ObjectId('69d61f44534bc077b27dde5c'),
  products: [
    ObjectId('69f4bee7562377cale05832d'),
    ObjectId('69f4bee7562377cale05832b'),
    ObjectId('69d62141534bc077b27dde5d'),
    ObjectId('69f4bee7562377cale05832c'),
    ObjectId('69f4bee7562377cale058330')
  ]
}
```

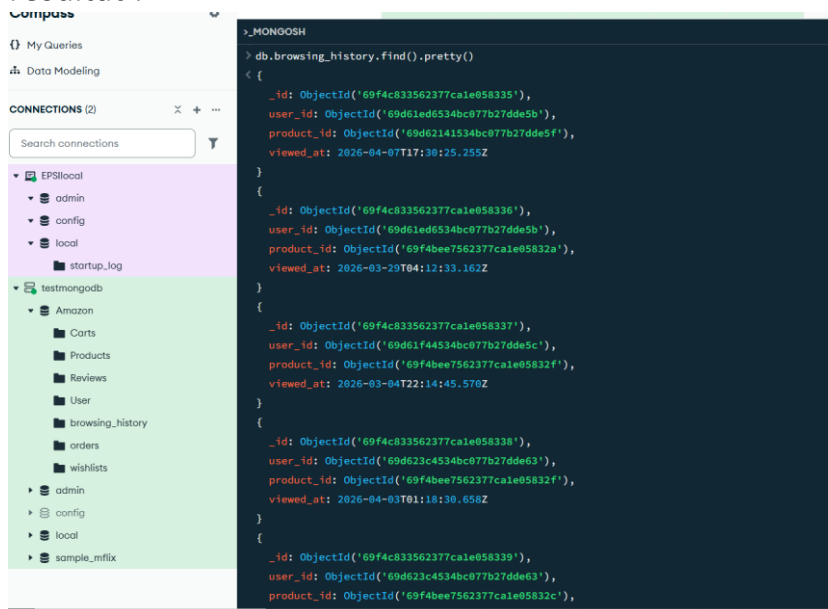
Exercice 2.2 : Historique de navigation

```
//Exercice 2.2 : Historique de navigation

const users = db.User.find({}, {_id: 1}).toArray()
//On récupère tous les utilisateurs de la collection User, les id
const products = db.Products.find({}, {_id: 1}).toArray()
//on récupère tous les produits avec uniquement leur _id

for (let i = 0; i < 35; i++) {
  db.browsing_history.insertOne({
    user_id: users[Math.floor(Math.random() * users.length)]._id,
    product_id: products[Math.floor(Math.random() * products.length)]._id,
    viewed_at: new Date(Date.now() - Math.random() * 1000 * 60 * 60 * 24 * 60)
  })
}
// générer 35 visites simulées
```

résultat :



Partie 3 — Requêtes d'agrégation

Exercice 3.1 : Chiffre d'affaires mensuel

Pipeline d'agrégation sur la collection orders pour calculer le chiffre d'affaires par mois. Seules les commandes au statut completed sont prises en compte.

```
//Actualiser le status de chaque order :

db.orders.updateMany(
  { status: "processing" },
  { $set: { status: "completed" } }
)

//tirage pas mois et annee

db.orders.aggregate([
  { $match: { status: "completed" } },// filtre pour orders avec un status completed
  { $project: { //On extrait le mois et l'année de chaque commande.
    month: { $month: "$order_date" },
    year: { $year: "$order_date" },
    total_amount: 1
  }},
  { $group: { // On regroupe par mois et on additionne les montants.
    _id: { year: "$year", month: "$month" },
    chiffre_affaires: { $sum: "$total_amount" }
  }},
  { $sort: { "_id.year": 1, "_id.month": 1 } } // trie les résultats chronologiquement.
])
```

resultat :

The screenshot shows the MongoDB Atlas Data Modeling interface. On the left, the 'CONNECTIONS (2)' panel lists two connections: 'EPSIlocal' and 'testmongodb'. The 'testmongodb' connection is expanded, showing a database named 'Amazon' with several collections: 'Carts', 'Products', 'Reviews', 'User', 'browsing_history', 'orders', and 'wishlists'. The 'orders' collection is selected. On the right, a MongoDB query is shown in a dark-themed editor. The query is an aggregation pipeline that filters for 'completed' orders, projects the month and year from the 'order_date' field, groups the results by month and year, and sums the 'total_amount' field. The results are sorted chronologically. The output of the query is displayed below the code, showing two documents: one for year 2026, month 2 with a total amount of 1539.96, and another for year 2026, month 3 with a total amount of 1629.95.

```
> db.orders.aggregate([
  { $match: { status: "completed" } },
  { $project: {
    month: { $month: "$order_date" },
    year: { $year: "$order_date" },
    total_amount: 1
  }},
  { $group: {
    _id: { year: "$year", month: "$month" },
    chiffre_affaires: { $sum: "$total_amount" }
  }},
  { $sort: { "_id.year": 1, "_id.month": 1 } }
])
< {
  _id: {
    year: 2026,
    month: 2
  },
  chiffre_affaires: 1539.96
}
{
  _id: {
    year: 2026,
    month: 3
  },
  chiffre_affaires: 1629.95
}
Atlas_atlas-13b0ze-shard-0 [primary] Amazon>
```

Exercice 3.2 : Produits souvent achetés ensemble

Ce pipeline identifie les paires de produits fréquemment co-commandés dans la même commande. Nous utilisons \$unwind pour décomposer les items, puis une auto-jointure avec \$lookup pour croiser les paires.

```

db.orders.aggregate([
  // 1. Décomposer le tableau items
  { $unwind: "$items" },

  // 2. Garder seulement _id de la commande et productId
  {
    $project: {
      _id: 1,
      productId: "$items.productId"
    }
  },

  // 3. Auto-jointure sur la même commande
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "_id",
      as: "same_order"
    }
  },

  // 4. Décomposer les items de la même commande
  { $unwind: "$same_order" },
  { $unwind: "$same_order.items" },

  // 5. Créer les paires
  {
    $group: {
      _id: {
        product1: { $min: ["$productId", "$same_order.items.productId"] },
        product2: { $max: ["$productId", "$same_order.items.productId"] }
      },
      count: { $sum: 1 }
    }
  },

  // 6. Éliminer les paires identiques (A, A)
  {
    $match: {
      $expr: { $ne: ["$_id.product1", "$_id.product2"] }
    }
  },

  // 7. Trier et limiter
  { $sort: { count: -1 } },
  { $limit: 10 }
])

```

```

// 7. Trier et limiter
{ $sort: { count: -1 } },
{ $limit: 10 }
])
< {
  _id: {
    product1: ObjectId('69d62141534bc077b27dde5e'),
    product2: ObjectId('69d62141534bc077b27dde5f')
  },
  count: 4
}
{
  _id: {
    product1: ObjectId('69d62141534bc077b27dde5d'),
    product2: ObjectId('69d62141534bc077b27dde5f')
  },
  count: 4
}
{
  _id: {
    product1: ObjectId('69d62141534bc077b27dde5d'),
    product2: ObjectId('69d62141534bc077b27dde5e')
  },
  count: 4
}
}

```

Partie 4 — Indexation

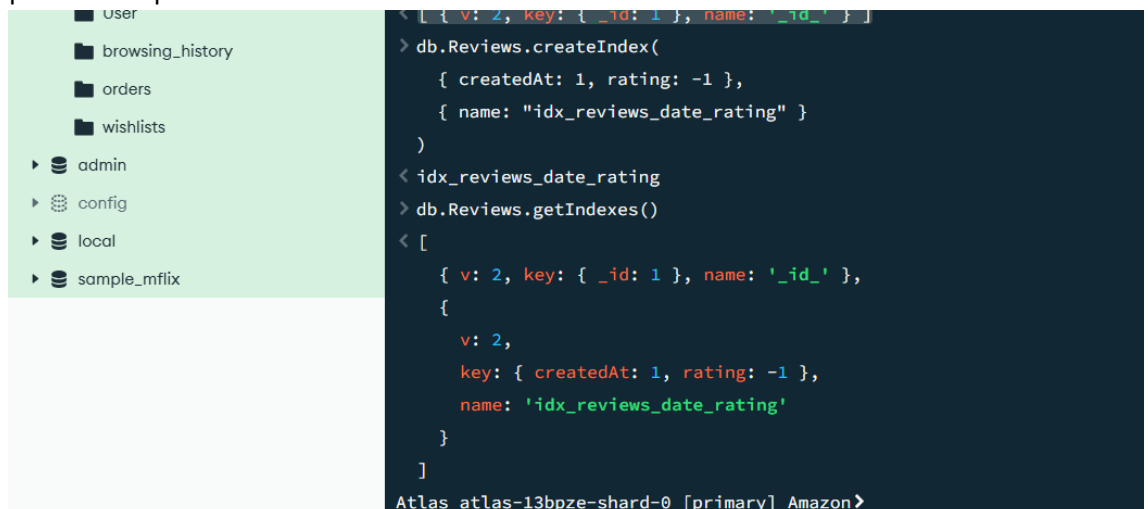
Exercice 4.1 : Index sur les avis (reviews)

Objectif : accélérer la requête du top 5 des produits.

Consignes :

Créer un index composé sur reviews pour optimiser

pour assure que on bien cree les index :



```

User
├── browsing_history
├── orders
├── wishlists
├── admin
├── config
├── local
└── sample_mflix

> use reviews
> db.Reviews.createIndex(
  { createdAt: 1, rating: -1 },
  { name: "idx_reviews_date_rating" }
)
< idx_reviews_date_rating
> db.Reviews.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { createdAt: 1, rating: -1 },
    name: 'idx_reviews_date_rating'
  }
]
Atlas atlas-13bpze-shard-0 [primary] Amazon>

```

Index proposé : { createdAt: 1, rating: -1 }

createdAt: 1(asc) en premier → car c'est le champ de filtre (Champ de filtre par date),

MongoDB l'utilise pour réduire les documents candidates

rating: -1(des) en second → car c'est le champ de tri décroissant

La requête à optimiser effectue deux opérations : un filtre sur la date (createdAt) et un tri décroissant sur la note (rating: -1).

Exercice 4.2 : Index pour l'historique de navigation

Objectif : faciliter la recherche des dernières visites d'un utilisateur.

Consignes :

Créer un index sur browsing_history pour :



The screenshot shows the MongoDB Atlas interface. On the left, a sidebar lists collections: browsing_history, orders, and wishlists. The main panel displays the command to create an index on the browsing_history collection:

```
db.browsing_history.createIndex(
  { user_id: 1, viewed_at: -1 },
  { name: "idx_browsing_userid_viewedat" }
)
```

Below the command, the output shows the index name: idx_browsing_userid_viewedat. At the bottom, the status bar indicates the connection is to Atlas atlas-13bpze-shard-0 [primary] Amazon.

```
db.browsing_history.createIndex(
  { user_id: 1, viewed_at: -1 },
  { name: "idx_browsing_userid_viewedat" }
)
```

user_id , 1(Filtre d'égalité) → toujours en **premier**

viewed_at , -1(Tri décroissant)→ dernières visites en premier

On crée un index composé sur user_id et viewed_at : d'abord le champ de **filtre** (user_id: 1) pour identifier l'utilisateur, puis le champ de **tri** (viewed_at: -1) pour obtenir les pages vues du plus récent au plus ancien. Cet ordre respecte la règle : champs de filtre avant champs de tri.